

Skype to initiate incoming calls. *Netcat* can thus work across simple NAT firewalls, and you can do a lot of very advanced networking functions with it, though *netcat* by itself does not break or add to security.

You can run a UDP server, instead of TCP, with *nc -u -l -p 1235* and you can run a UNIX domain server with *nc -U -l /tmp/unixsock*, which shows you that many really great things can be done with it. With experience working on UNIX, and an understanding of networking, you can unlock *netcat*'s potential.

You can do a simple file transfer between two machines; on machine A, run *nc -l -p 1234 > /tmp/newfile* and on machine B, run *cat newfile | nc -v remoteip 1234*, which fetches *newfile* from B to A. Interesting, right?

As I said, you can easily script *netcat*, since it does not have the interactive password requirements of SSH. In fact, you can also marry *netcat* and SSH and obtain a simple TCP proxy. But all that requires another more comprehensive article. For now, we are done with this great tool. Let us now talk about the *pf* firewall found in OpenBSD.

The OpenBSD *pf* firewall

OpenBSD is not the only OS to have the *pf* firewall. Though originally designed and written for that OS, today we find *pf* in FreeBSD and NetBSD too. In fact, the popular *pfSense* ISO is based on OpenBSD *pf* but it runs FreeBSD.

FreeBSD *pf* is not the real thing. OpenBSD is where *pf* is natively developed and maintained; FreeBSD ports it later, so you are better off running it natively. Besides, *pf*, being firewall software, uses kernel networking code extensively. OpenBSD *pf* is also tightly integrated with its own kernel networking sub-system. Consequently, *pf* works best on OpenBSD.

Now, like Linux *iptables*, the *pf* firewall can do a lot of cool stuff like port forwarding. However, *iptables* is very complex, and does too many things. Writing to payload data in protocols like FTP, SIP, RTP, etc, is something that user-land proxy applications do. That is not the job of a kernel firewall. Also, content inspection is not its job. A kernel firewall like *pf* limits itself only to packet headers at TCP or Layer IV. Beyond that, user-land applications have to handle the data. However, *pf* is very powerful, since it can do things like direct server return, and divert-to-sockets, which can be used for very advanced packet forwarding, routing, and so on.

All *pf* rules are normally written in */etc/pf.conf*, but can be added from the command line or included from other files as well. However, in general, keeping all firewall rules simple, short and sweet is the preferred convention. In OpenBSD, everything is kept very simple; that is why it is so secure. And *pf* rules read

just like plain English – they are so easy to read that you will forget you are writing a computer language. Still, firewall rules take a lot of learning and it's not possible to cover them in one article. I have worked very hard to learn *pf* rules, and despite using them extensively in all our products, even today I have difficulty with certain rules. It takes a lot of time and experience to master them.

Now, let us look at a simple NAT set-up rule, in */etc/pf.conf*:

```
pass out quick on egress from 192.168.0.0/16 to any nat-to
2.3.4.5
```

The public interface IP is assumed to be 2.3.4.5, and the local network is 192.168.0.0/16.

For new or changed *pf* rules, you load the rules using (as root) the *pfctl -f /etc/pf.conf* command. If the firewall is not enabled, you enable it with *pfctl -e* while you can view active rules with *pfctl -sr* (the *-sr* stands for *show rules*).

Pf can do many things like simple TCP and UDP port forwarding, for instance. A simple TCP port forwarding rule to forward the SMTP port on the public interface of the firewall to a local machine 192.168.1.5 is set up as follows:

```
pass in inet proto tcp from any to egress port 25 rdr-to
192.168.1.5
```

Remember, port forwarding is messy to get right. The reverse of NAT, it involves state maintenance and tracking. So unless symmetric routing is enforced, in which the upstream and downstream paths of the packets are going through the same firewall, things don't work as expected. The TCP handshake itself will fail and you can easily check with *netcat* that the port forwarding does not work as intended. You can easily forward a range of ports. For example, to forward ports 5000 to 6000, both inclusive, issue the following command:

```
pass in inet proto tcp from any to egress port 5000:6000 rdr-to
192.168.1.5
```

You can make that a UDP rule by simply changing *proto tcp* to *proto udp* in that line.

Now, if you found that easy, this is what *pf* is all about; simple to use, yet powerful enough for sophisticated tasks. 

By: Girish Venkatachalam

The author runs a company named Gayatri Hitech (<http://gayatri-hitech.com>) that creates computer networking products like firewalls, mail servers, VPNs, etc. He may be contacted at girish@gayatri-hitech.com. He is on Twitter, Skype and GMail as *girish1729*.